9

Лекция 6–7 апреля

Машина Тьюринга

Константин Чухарев

Дискретная математика — Весна 2027

Машина Тьюринга

> *Вычисление обычно выполняется записыванием определённых символов на бумаге.*
>
> ***— Алан Тьюринг***

Чего не хватило автомату?

Язык $a^n b^n$ нерегулярен, и доказано это дважды, леммой о накачке и теоремой Майхилла–Нерода.

Причина честная — у автомата только конечное число состояний, и бесконечный счёт в них не помещается.

> Люди считают карандашом на бумаге, стирая и переписывая написанное.
>
> Модель вычисления обязана уметь то же самое — запоминать неограниченно много.

Человек за столом

Тьюринг в 1936 году описал вычисление так, как его делает человек с карандашом.

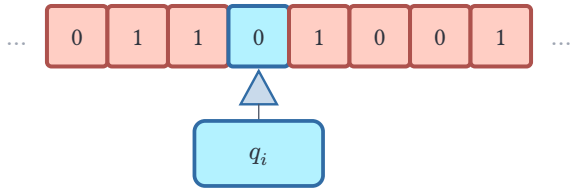
- конечное число «состояний души» — то, что человек помнит
- бумага, разбитая на клетки — то, куда он смотрит
- простые шаги — стереть, написать, сдвинуть взгляд.

Здесь нет никакой магии, есть только символы, клетки и правила.

Всё остальное вычисление получает из их комбинаций.

Тьюринг описал вычислителя, а не вычислительную машину. Под вопросом была проблема разрешения Гильберта, а чтобы доказать отсутствие алгоритма, алгоритм сначала нужно было определить.

Устройство машины



У трёх частей чёткие роли. Бесконечная лента служит памятью, головка — взглядом, конечное управление — программой.

Бесконечность ленты означает отсутствие предела, а не готовый склад. В каждый момент занята конечная часть, и любое вычисление касается лишь конечного числа ячеек.

Семёрка машины Тьюринга

Три части складываются в один формальный объект. Конечная часть объекта невелика, это состояния, алфавиты и таблица правил.

Определение 1 (Машина Тьюринга)

Машина задаётся семёркой $M = \big(Q, \Gamma, \Sigma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\big)$.

- Q — конечное множество состояний.
- Γ — алфавит ленты, содержит входной алфавит Σ и пробел.
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ — частичная функция переходов.
- q_0 — начальное состояние, q_{accept} и q_{reject} — принимающее и отвергающее.

Состояния конечны, а лента нет. Весь неограниченный счёт живёт на ленте.

Роли компонентов и границы

Компонент	Роль	Оговорка
Q	память между шагами, состояния помнят, что видели	конечна — счёт в состояниях не помещается
Γ, Σ	алфавит ленты с пробелом и входной алфавит	на входе пробелов нет, вокруг — пробелы
δ	пара (состояние, символ) даёт тройку	частична, нет правила — остановка
$q_0, q_{\text{accept}}, q_{\text{reject}}$	старт и два различимых финала	финалы различны, иначе ответ без знака

Каждая оговорка — это выбор. Расширения не меняют класс, а частичность δ делает остановку частью модели.

Один шаг

Вся работа машины складывается из атомарных шагов, и шаг — это всегда срабатывание одного правила.

Определение 2 (Переход)

На паре (состояние, символ) правило δ даёт тройку (новое состояние, новый символ, сдвиг). Машина записывает новый символ под головкой, сдвигает головку влево или вправо и переходит в новое состояние.

Функция δ частична, поэтому правила для текущей пары может не быть.

Остановка вне финальных состояний по соглашению считается отверганием.

Конфигурация

Определение 3 (Конфигурация)

Мгновенное описание работы задаётся записью uqv.

- u — содержимое ленты слева от головки
- q — текущее состояние
- v — лента от головки вправо, включая читаемую ячейку.

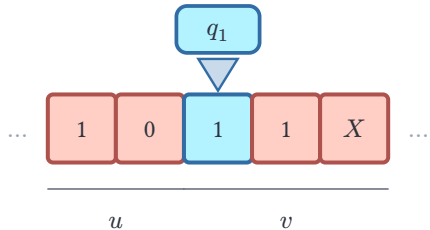
Начальная конфигурация на входе w — это $q_0 w$. Головка стоит на первом символе.

Пример (Чтение конфигурации)

В записи $1q_0010$ имеем $u = 1$, состояние q_0 и $v = 010$.

Головка стоит на первом символе v и читает 0.

Анатомия конфигурации



На схеме лента хранит $1011X$, головка стоит на третьей ячейке, и состояние q_1 нарисовано прямо над ней. Скобки внизу показывают, какие ячейки читаются как u и как v. Слева от головки лежит u, от головки вправо — v. Пустые ячейки за X в запись не попадают, поэтому конфигурация всегда конечна, какой бы длинной ни была занятая часть ленты.

Три исхода

Определение 4 (Исходы работы)

- МТ **принимает** вход, если достигает q_{accept}.
- МТ **отвергает**, если достигает q_{reject}.
- МТ может **зациклиться** — не достигнуть ни того, ни другого.

Третий исход — часть модели, а не каприз определения.

Программа, которая крутится вечно, должна описываться честно.

У автомата два исхода. Вход читается до конца, и ответ обязан появиться.

Машина Тьюринга добавляет молчание — ответа не будет ни на каком шаге.

Что значит решить язык?

Определение 5 (Распознаваемость и разрешимость)

Язык L **распознаваем**, если есть МТ, принимающая w в точности тогда, когда $w \in L$.

На $w \notin L$ машина может отвергнуть или зациклиться.

Язык L **разрешим**, если есть МТ, которая всегда останавливается. Такая машина принимает $w \in L$ и отвергает $w \notin L$.

Разрешимый язык распознаваем, а обратное — нет.

Разрешимость требует остановки на каждом входе, а распознавание — только приёма слов языка.

Слова, заканчивающиеся на ноль

Пример (Семёрка в деле)

Рассмотрим язык слов над $\{0, 1\}$, заканчивающихся на 0.

Два состояния различают последний прочитанный символ. Состояние q_0 помнит 1, состояние q_1 помнит 0.

- $(q_0, 1) \to (q_0, 1, R)$
- $(q_0, 0) \to (q_1, 0, R)$
- $(q_0, \text{пробел}) \to (q_{\text{reject}}, \text{пробел}, R)$
- $(q_1, 0) \to (q_1, 0, R)$
- $(q_1, 1) \to (q_0, 1, R)$
- $(q_1, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$

Лента тут только возит головку, язык регулярный, и памяти хватает двух состояний. Лента понадобится уже для подсчёта.

Полная трасса по шагам

Прогоним машину из примера выше на входе 1010, записывая каждый шаг отдельной строкой.

Пример (Шаг за шагом)

Шаг	Конфигурация	Сработавшее правило
0	$q_0 1010$	$(q_0, 1) \to (q_0, 1, R)$ — читаем 1, шаг вправо
1	$1 q_0 010$	$(q_0, 0) \to (q_1, 0, R)$ — ноль запомнен
2	$10 q_1 10$	$(q_1, 1) \to (q_0, 1, R)$ — единица сменила память
3	$101 q_0 0$	$(q_0, 0) \to (q_1, 0, R)$ — снова ноль в состоянии
4	$1010 q_1$ пробел	$(q_1, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$ — приём на краю

Конфигурация $q_0 1010$ переходит в $1 q_0 010$ по одному правилу, и вся работа машины — цепочка таких переходов.

Трасса и разрешимость

Пример (Трасса на 10)

$$q_0 10 \to 1 q_0 0 \to 10 q_1 \text{ пробел} \to 10 q_{\text{accept}} \text{ пробел}.$$

На последнем шаге q_1 видит пробел и переходит в q_{accept}.

Головка сдвигается только вправо, поэтому машина делает ровно $|w| + 1$ шаг и останавливается на любом входе.

Значит, язык разрешим, и память машины здесь не растёт.

Программы на ленте

> *Программы должны писаться для того, чтобы их читали люди, и лишь попутно — чтобы их исполняли машины.*
>
> ***— Гарольд Абельсон и Джеральд Сассман***

Инверсия битов

Пример (Первый алгоритм)

Заменить каждый 0 на 1 и каждую 1 на 0.

Хватает одного состояния и трёх правил.

- $(q_0, 0) \to (q_0, 1, R)$
- $(q_0, 1) \to (q_0, 0, R)$
- $(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$

Эта машина вычисляет функцию, а не распознаёт язык. На ленте остаётся результат.

Разрешимость — про ответ «да или нет», вычислимость — про выход.

Правилам не нужно смотреть на соседей. Каждое решение принимается по паре (состояние, символ), и глобальный эффект складывается из локальных шагов.

Инкремент двоичного числа

Пример (Прибавить единицу)

Число записано на ленте, младший бит слева.

Пока читаем 1 — пишем 0 и уносим перенос вправо.

Первая 0 меняется на 1, и работа закончена.

- $(q_0, 1) \to (q_0, 0, R)$
- $(q_0, 0) \to (q_{\text{accept}}, 1, R)$
- $(q_0, \text{пробел}) \to (q_{\text{accept}}, 1, R)$

Лента работает как регистр неограниченной длины. Новый разряд дописывается сам, и переполнения не бывает.

Трасса инкремента

Пример (Трасса на 11)

1. Шаг в $q_0 11$ читает младшую 1, пишет 0 и уносит перенос вправо.
2. Шаг в $0 q_0 1$ снова видит 1, пишет 0 и уносит перенос вправо.
3. Шаг в $00 q_0$ пробел видит пробел, пишет 1 и принимает.

Число 3 выросло на единицу и записано как 001. Младший бит снова слева.

Все единицы исчерпаны — машина дописала новый разряд.

Лента сама расширяет число, и ничего перевыделять не нужно.

Сдвиг слова на клетку влево

Слово переезжает на клетку влево, и справа остаётся свободное место.

Пример (Перенос по одному символу)

Роли состояний распределены так. q_{read} читает очередной символ, q_{carry0} и q_{carry1} несут его на клетку влево, q_{hop} возвращается вправо и затирает источник.

- $(q_{\text{read}}, 0) \to (q_{\text{carry0}}, 0, L)$, $(q_{\text{read}}, 1) \to (q_{\text{carry1}}, 1, L)$
- $(q_{\text{carry0}}, \text{пробел}) \to (q_{\text{hop}}, 0, R)$, $(q_{\text{carry1}}, \text{пробел}) \to (q_{\text{hop}}, 1, R)$
- $(q_{\text{hop}}, 0) \to (q_{\text{read}}, \text{пробел}, R)$, $(q_{\text{hop}}, 1) \to (q_{\text{read}}, \text{пробел}, R)$
- $(q_{\text{read}}, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$

Клетка слева всегда свободна. У первого символа она свободна изначально, у остальных её освободил предыдущий шаг.

Трасса сдвига

Прогон на входе 10 показывает, как слово сдвигается влево, а справа освобождается клетка.

Пример (Слово уезжает влево)

Шаг	Конфигурация	Сработавшее правило
0	$q_{\text{read}}10$	$(q_{\text{read}}, 1) \to (q_{\text{carry1}}, 1, L)$ — единица запомнена
1	q_{carry1} пробел 10	$(q_{\text{carry1}}, \text{пробел}) \to (q_{\text{hop}}, 1, R)$ — слева свободно, пишем
2	$1q_{\text{hop}}10$	$(q_{\text{hop}}, 1) \to (q_{\text{read}}, \text{пробел}, R)$ — источник стёрт
3	1 пробел $q_{\text{read}}0$	$(q_{\text{read}}, 0) \to (q_{\text{carry0}}, 0, L)$
4	$1q_{\text{carry0}}$ пробел 0	$(q_{\text{carry0}}, \text{пробел}) \to (q_{\text{hop}}, 0, R)$
5	$10q_{\text{hop}}0$	$(q_{\text{hop}}, 0) \to (q_{\text{read}}, \text{пробел}, R)$
6	10 пробел q_{read} пробел	$(q_{\text{read}}, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$ — слово кончилось

Копирование слова

Копирование — блок, без которого не собирается большая программа.

Пример (Пометка и разделитель)

Лента работает с алфавитом $\Gamma = \{0, 1, \hat{0}, \hat{1}, Y, \text{пробел}\}$.

- q_{start} идёт вправо до пробела и ставит Y
- q_{left} возвращается к левому краю слова
- q_{pick} находит первый непомеченный символ, помечает его и запускает копирование. На Y — конец копирования.
- q_{copy0}, q_{copy1} несут символ до конца копии, дописывают его и возвращаются к q_{pick}, не снимая шляпку
- q_{restore} идёт влево, снимая шляпки, только после того, как все символы скопированы
- q_{clean} проходит вправо и принимает на пробеле

Лента копирования по фазам

На вход подаётся слово из единственного символа.

Лента проходит фазы так.

Пример (От входа до копии)

$$1 \to 1\mathrm{Y} \to \hat{1}\ \mathrm{Y} \to \hat{1}\ \mathrm{Y}1 \to 1\mathrm{Y}1$$

Маркер Y отделяет копию, шляпка помечает прочитанный символ. Для слова из нескольких символов шляпки остаются до конца копирования. q_{restore} снимает их все сразу.

Копия строится слева направо. Каждый проход несёт один символ до конца копии и возвращается назад.

Время — $O(n^2)$ шагов, занятая часть ленты — $O(n)$.

Язык $a^n b^n$ на ленте

Пример (Вычёркивание пар)

На язык $a^n b^n$ конечному автомату не хватило состояний.

Каждый проход вычёркивает один a слева и одну b справа.

Лента на входе aabb после первого прохода и после второго выглядит так.

$$\mathrm{aabb} \to \mathrm{XabX} \to \mathrm{XXXX}.$$

Когда вся лента покрыта маркерами, машина видит пробел и принимает.

Конечному автомату не хватило состояний на счёт, а машине хватает ленты.

Сравнить два счётчика — значит стереть их попарно.

Таблица переходов для вычёркивания пар

Пример (Машина для $a^n b^n$)

Лента работает с алфавитом $\Gamma = \{a, b, X, \text{пробел}\}$. Состояние q_0 ищет первый невычеркнутый символ, q_1 идёт вправо, q_2 ищет последнюю b, q_3 возвращается.

- $q_0 - (q_0, X) \to (q_0, X, R)$, $(q_0, a) \to (q_1, X, R)$, $(q_0, b) \to \big(q_{\text{reject}}, b, R\big)$, $(q_0, \text{пробел}) \to \big(q_{\text{accept}}, \text{пробел}, R\big)$
- $q_1 - (q_1, a) \to (q_1, a, R)$, $(q_1, b) \to (q_1, b, R)$, $(q_1, X) \to (q_1, X, R)$, $(q_1, \text{пробел}) \to (q_2, \text{пробел}, L)$
- $q_2 - (q_2, X) \to (q_2, X, L)$, $(q_2, b) \to (q_3, X, L)$, $(q_2, a) \to \big(q_{\text{reject}}, a, L\big)$, $(q_2, \text{пробел}) \to \big(q_{\text{reject}}, \text{пробел}, R\big)$
- $q_3 - (q_3, a) \to (q_3, a, L)$, $(q_3, b) \to (q_3, b, L)$, $(q_3, X) \to (q_0, X, R)$

Три ловушки закрывают все несовпадения. Лишняя b ловится в q_0, a после b — в q_2 на символе, лишние a — поиском b до пробела.

Проход 1, шаги 0–4

Полная трасса на aabb, конфигурация за конфигурацией.

Пример (Вычеркнуть первый a и дойти до края)

Шаг	Конфигурация	Сработавшее правило
0	q_0 aabb	$(q_0, a) \to (q_1, X, R)$ — вычеркнули первый a
1	Xq_1 abb	$(q_1, a) \to (q_1, a, R)$
2	Xa q_1 bb	$(q_1, b) \to (q_1, b, R)$
3	Xab q_1 b	$(q_1, b) \to (q_1, b, R)$
4	Xabb q_1 пробел	$(q_1, \text{пробел}) \to (q_2, \text{пробел}, L)$ — край достигнут

Возврат, шаги 5–8

С конца вычёркивается последняя b, головка возвращается к границе.

Пример (Вычеркнуть последнюю b и вернуться)

Шаг	Конфигурация	Сработавшее правило
5	Xab q_2 b	$(q_2, b) \to (q_3, X, L)$ — вычеркнули последнюю b
6	Xa q_3 bX	$(q_3, b) \to (q_3, b, L)$ — возврат влево
7	Xq_3 abX	$(q_3, a) \to (q_3, a, L)$
8	q_3 XabX	$(q_3, X) \to (q_0, X, R)$ — граница, к поиску

Проход 2, шаги 9–14

Осталось два невычеркнутых символа — один a и одна b.

Пример (Вычеркнуть оставшуюся пару)

Шаг	Конфигурация	Сработавшее правило
9	Xq_0 abX	$(q_0, a) \to (q_1, X, R)$ — второй проход начался
10	XX q_1 bX	$(q_1, b) \to (q_1, b, R)$
11	XXb q_1 X	$(q_1, X) \to (q_1, X, R)$
12	XXbX q_1 пробел	$(q_1, \text{пробел}) \to (q_2, \text{пробел}, L)$
13	XXb q_2 X	$(q_2, X) \to (q_2, X, L)$ — вычеркнутый хвост
14	XX q_2 bX	$(q_2, b) \to (q_3, X, L)$ — последняя b вычеркнута

Приём, шаги 15–18

Пример (Приём)

Шаг	Конфигурация	Сработавшее правило
15	$X q_3$ XXX	$(q_3, X) \to (q_0, X, R)$ — граница, к поиску
16	XX q_0 XX	$(q_0, X) \to (q_0, X, R)$ — пропуск маркеров
17	XXX q_0 X	$(q_0, X) \to (q_0, X, R)$
18	XXXX q_0 пробел	$(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$ — всё вычеркнуто
19	XXXX q_{accept}	принято, $n = 2$, как и было

Каждый проход уменьшает невычеркнутую часть ровно на два символа, поэтому работа кончается. Финал — либо приём, либо одно из отвергающих правил.

Остановка гарантирована на любом входе — таблица описывает решатель.

Палиндромы

Пример (Сравнение концов)

Палиндром над $\{a, b\}$ одинаково читается в обе стороны, например abba и bab.

Стратегия состоит в том, чтобы сравнить крайние символы, стереть оба и повторять, пока строка не кончится.

Состояние q_0 читает с начала, q_a и q_b несут символ вправо, p_a и p_b сверяют его с правым краем, q_{back} возвращает головку.

При сверке возможны три исхода. Совпадение стирает правый символ, несовпадение отвергает, пробел слева указывает на середину нечётного слова.

МТ видит оба конца строки ценой многократных проходов. Запоминание через состояние — то же, что у автомата, но возвраты делает головка.

Отвергание палиндрома

Сверка кончается при первом же несовпадении, и остаток слова машина не читает.

Пример (Несовпадение на входе ab**)**

$$q_0\ \mathrm{ab} \to X q_a\ \mathrm{b} \to \mathrm{Xb}\ q_a\ \text{пробел} \to X p_a\ \mathrm{b} \to \mathrm{Xb}\ q_{\mathrm{reject}}\ \text{пробел}.$$

Запомненный символ несёт состояние. Состояние p_a сверяет с a, p_b — с b.

Проход 1 палиндрома

Полный прогон на abba занимает по строке на шаг, а маркер X помечает стёртую ячейку.

Пример (Запомнить a и сверить правый край)

Шаг	Конфигурация	Сработавшее правило
0	q_0 abba	$(q_0, a) \to (q_a, X, R)$ — a стёрт и запомнен
1	Xq_a bba	$(q_a, b) \to (q_a, b, R)$
2	Xb q_a ba	$(q_a, b) \to (q_a, b, R)$
3	Xbb q_a a	$(q_a, a) \to (q_a, a, R)$
4	Xbba q_a пробел	$(q_a, \text{пробел}) \to (p_a, \text{пробел}, L)$ — край, развернуться
5	Xbb p_a a	$(p_a, a) \to (q_{\text{back}}, X, L)$ — край совпал, стереть
6	Xb q_{back} bX	$(q_{\text{back}}, b) \to (q_{\text{back}}, b, L)$ — возврат влево

Возврат и второй заход палиндрома

q_{back} доходит до границы, q_0 стирает следующий символ, и машина сверяет второй край.

Пример (Стереть b и развернуться у правого края)

Шаг	Конфигурация	Сработавшее правило
7	Xq_{back} bbX	$(q_{\text{back}}, b) \to (q_{\text{back}}, b, L)$
8	q_{back} XbbX	$(q_{\text{back}}, X) \to (q_0, X, R)$ — граница, к чтению
9	Xq_0 bbX	$(q_0, b) \to (q_b, X, R)$ — b стёрт и запомнен
10	XX q_b bX	$(q_b, b) \to (q_b, b, R)$
11	XXb q_b X	$(q_b, X) \to (q_b, X, R)$ — мимо стёртого хвоста
12	XXbX q_b пробел	$(q_b, \text{пробел}) \to (p_b, \text{пробел}, L)$ — край, развернуться
13	XXb p_b X	$(p_b, X) \to (p_b, X, L)$ — пропуск стёртых

Приём палиндрома

Последняя пара стирается, и на ленте остаются только маркеры.

Пример (Второе сравнение и приём)

Шаг	Конфигурация	Сработавшее правило
14	XX p_b bX	$(p_b, b) \to (q_{\text{back}}, X, L)$ — край совпал, стереть
15	$X q_{\text{back}}$ XXX	$(q_{\text{back}}, X) \to (q_0, X, R)$ — граница, к чтению
16	XX q_0 XX	$(q_0, X) \to (q_0, X, R)$ — пропуск маркеров
17	XXX q_0 X	$(q_0, X) \to (q_0, X, R)$
18	XXXX q_0 пробел	$(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$ — всё стёрто
19	XXXX q_{accept}	принято, abba — палиндром

Два прохода на пару дают $O(n^2)$ шагов, многоленточной машине хватает $O(n)$.

Приёмы программирования

Примеры собираются из повторяющихся приёмов.

Приём	**Идея**
Запоминание через состояние	«вижу символ a» — перейди в состояние «помню a» и неси его, пока не понадобится
Маркеры	обработанный символ заменяется на X, и «уже учтено» отделяется от «ещё нет»
Проход к краю и возврат	«иди вправо, пока не пробел», шаг влево, переключение — цикл с условием на краю

Счёт на МТ — это всегда расстановка и пересчёт маркеров.

Память машины растёт не числом состояний, а занятой частью ленты.

Композиция машин

Машины собираются друг из друга. Выход одного блока становится входом следующего.

Определение 6 (Композиция)

Если M_1 переводит u в v, а M_2 переводит v в w, состояния останова M_1 и старта M_2 отождествляются. Перед запуском M_2 головку ставят в позицию, с которой M_2 умеет начинать. При необходимости между блоками добавляют возврат к левому краю.

Из блоков собираются большие программы.

Пример (Собранная машина)

Составная машина удваивает унарное число и инвертирует биты, вычисляя $1^n \to 0^{2n}$.

На входе 11 удвоитель даёт 1111, инвертор — 0000.

Останов составной машины — останов последнего блока.

Машина как данные

Правило перехода — это данные, пара на входе и тройка на выходе. Таблицу переходов можно положить в обычную структуру данных.

Пример (Таблица на Rust)

```rust
fn step(q: &str, a: char) -> Option<(char, isize, &'static str)> {
    match (q, a) {
        ("q0", '1') => Some(('1', 1, "q0")),
        ("q0", '0') => Some(('0', 1, "q1")),
        ("q0", '#') => Some(('#', 1, "rej")),
        ("q1", '1') => Some(('1', 1, "q0")),
        ("q1", '0') => Some(('0', 1, "q1")),
        ("q1", '#') => Some(('#', 1, "acc")),
        _ => None,
    }
}
```

Цикл интерпретатора с головкой

Пример (Печать конфигурации uqv на каждом витке)

```
fn main() {
    let mut tape: Vec<char> = "1010#".chars().collect();
    let (mut q, mut i) = ("q0", 0);
    while q != "acc" && q != "rej" {
        let u: String = tape[..i].iter().collect();
        let v: String = tape[i..].iter().collect();
        println!("{}", format!("{u} {q} {v}").trim_start());
        let Some((b, d, q2)) = step(q, tape[i]) else { break };
        tape[i] = b;
        i = (i as isize + d) as usize;
        q = q2;
    }
}
```

Что печатает симулятор?

Пример (Запуск на 1010**)**

```
q0 1010#
1 q0 010#
10 q1 10#
101 q0 0#
1010 q1 #
```

Пять строк — пять шагов таблицы переходов, и цикл вышел в acc.

Роль головки играет индекс i, а символ # изображает пробел. У вектора нет бесконечности, край помечен явно.

Машина как процессор

Приёмы, из которых собраны программы, — прообраз настоящего процессора.

> Состояние играет роль счётчика команд, маркер — флаг, проход по ленте — цикл.
>
> Таблица переходов конечна, и её можно выписать строкой символов. Программу удаётся положить на ленту и читать как данные.

Отсюда один шаг до машины, которая исполняет любую программу.

Машина как модель

Лента даёт неограниченную память при конечном описании, а головка позволяет вернуться к любой уже прочитанной ячейке.

- МТ — бесконечная лента плюс конечное управление, то есть неограниченная память при финитном описании.
- Конфигурация uqv — полный снимок работы, вычисление — цепочка конфигураций, исходов три.
- Программы пишутся приёмами, такими как символ в состоянии, маркеры, проходы и композиция, а лента заменяет счётчик, поэтому $a^n b^n$ разрешимо.
- Таблица переходов конечна, поэтому её исполняет обычный цикл, а сама машина оказывается данными.

Вопросы по машинам Тьюринга

о машине Тьюринга

1. Почему $a^n b^n$ нерегулярен, но разрешим машиной Тьюринга?
2. Назовите все семь компонентов семёрки МТ и роль каждого.
3. Что записывает конфигурация uqv и как выглядит начальная конфигурация для входа 10?
4. Чем распознаваемость языка отличается от разрешимости?
5. Как машина сдвигает слово на клетку влево и почему несущее состояние всегда попадает на пробел?
6. Почему таблицу переходов можно исполнить обычным циклом и что в этом цикле играет роль головки?

Что почитать?

Задача	Где читать
Понять МТ	m25, раздел «Определение машины Тьюринга»
Писать программы	m25, раздел «Примеры машин Тьюринга»
Свести приёмы в таблицу	m25, раздел «Стандартные приёмы программирования»